

Comprehensive C++ Training Guide

Sections 18–20: File I/O, Exception Handling & Namespaces

Target Audience: Beginners with no prior programming experience

Depth: Beginner → Expert

Format: 3-Day Intensive Reference (Day 3 — Advanced Topics)

TABLE OF CONTENTS

- [Section 18: File I/O and File Handling](#)
 - [18.1 File I/O Concepts](#)
 - [18.2 Streams: ofstream, ifstream, fstream](#)
 - [18.3 File Position Pointers](#)
- [Section 19: Exception Handling](#)
 - [19.1 Basics of Exception Handling](#)
 - [19.2 try, catch, and throw Keywords](#)
 - [19.3 Standard Exceptions](#)
 - [19.4 Uncaught Exceptions](#)
- [Section 20: Namespaces](#)
 - [20.1 Understanding Namespaces](#)
 - [20.2 Rules of Namespaces](#)

Section 18: File I/O and File Handling

18.1 File I/O Concepts

Learning Objectives

- Understand what File I/O means and why persistent storage matters
- Learn how C++ models files as streams of bytes
- Distinguish between text mode and binary mode
- Open, read, write, and close files safely
- Handle file errors using state flags

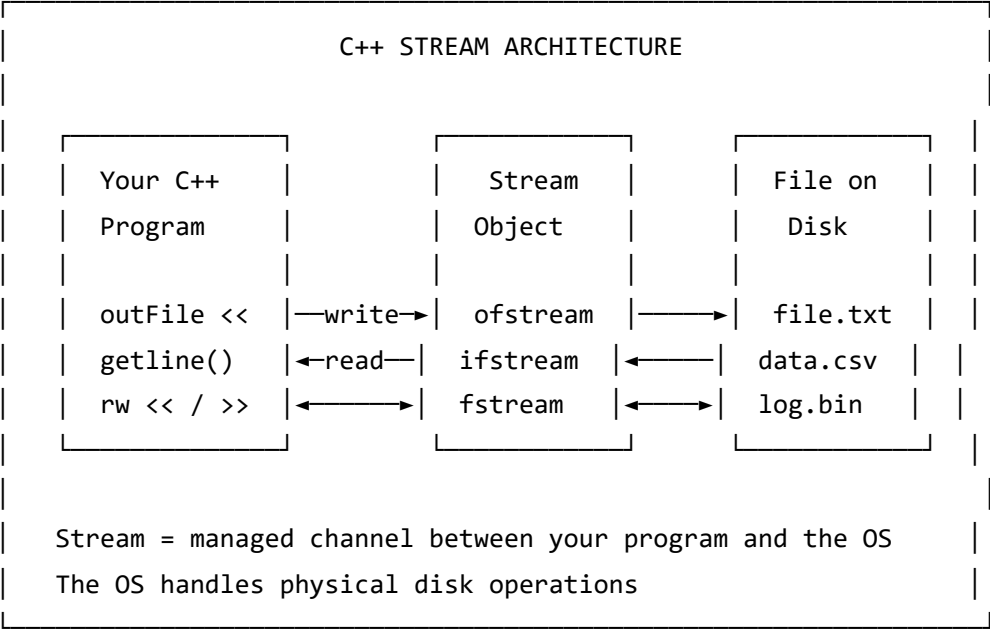
Introduction

Every program you've written so far stores data in RAM — the moment the program closes, all data vanishes. **File I/O** (Input/Output) solves this fundamental problem by allowing your program to read from and write to files on disk, making data **persistent** across program runs.

Think of file I/O as a postal service between your program and the operating system. Your program writes a letter (data), hands it to the OS (open a stream), the OS delivers it to the right address (file path), and stores it permanently. Reading is the reverse: the OS fetches the letter and delivers it back to your program.

In C++, all file operations are modeled as **streams** — a sequence of bytes flowing in one direction or both. This mirrors how you already use `std::cout` (a stream to the terminal) and `std::cin` (a stream from the keyboard). File streams work identically, but they connect to disk files instead.

The Stream Model — Architecture Diagram



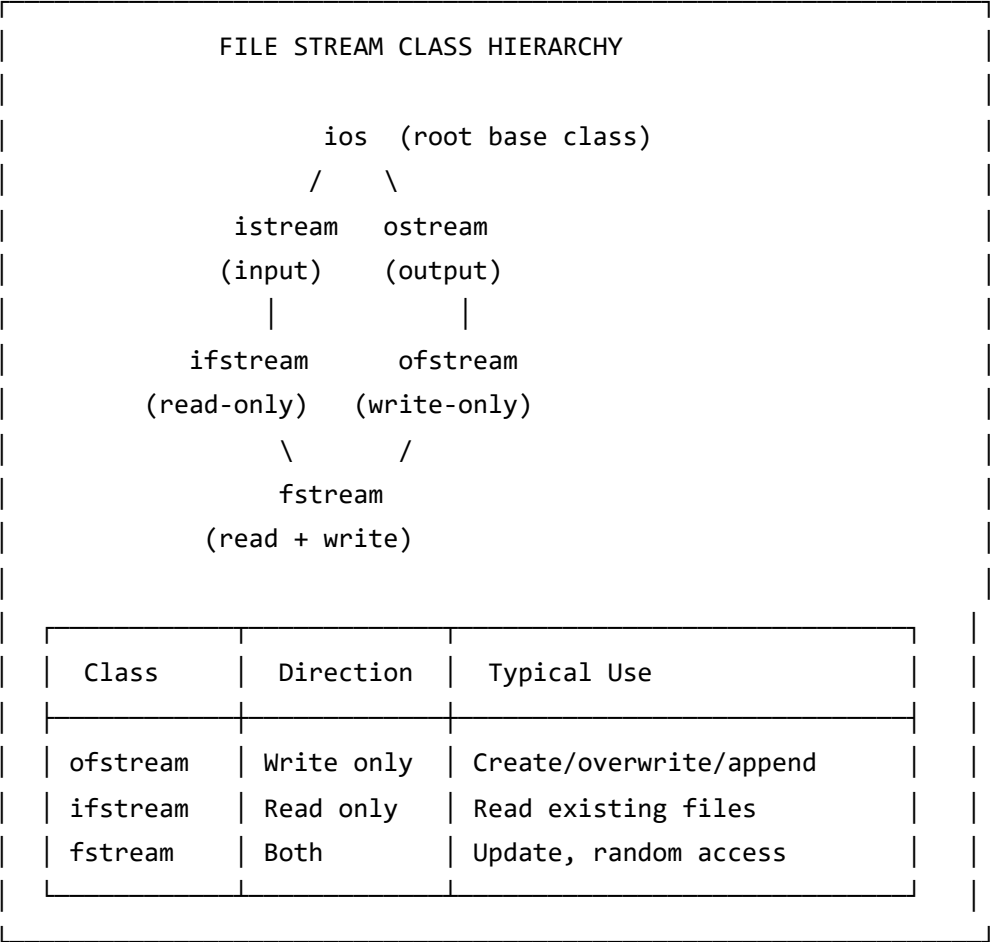
Text Mode vs Binary Mode — Comparison Table

TEXT MODE vs BINARY MODE	
TEXT MODE	BINARY MODE
Human-readable output	Raw byte sequences
'\n' translated on Windows (\n ↔ \r\n)	No byte translation ever
Default for all streams	Requires ios::binary flag
Best for: .txt, .csv, .html, .json	Best for: images, audio, structs, executables, serialized data
Line counting works	Must track byte offsets manually

18.2 Streams

C++ provides three file stream classes in <fstream> :

Class Hierarchy Diagram



Flag	Effect
ios::ate	Seek to end immediately after open

Example 1 — Basic File Writing

```
#include <iostream>
#include <fstream>    // Required for all file stream types
#include <string>

int main() {
    // — Step 1: Open a file for writing —
    // Constructor opens the file; if it doesn't exist, it is CREATED.
    // If it already exists, it is OVERWRITTEN (ios::trunc is default).
    std::ofstream outFile("hello.txt");

    // — Step 2: Always verify the file opened successfully —
    if (!outFile.is_open()) {
        std::cerr << "ERROR: Could not open hello.txt for writing.\n";
        std::cerr << " Possible causes: permission denied, disk full, bad path\n";
        return 1;    // Exit with error code
    }

    // — Step 3: Write to the file using << (same syntax as cout) —
    outFile << "Hello, File World!\n";
    outFile << "This is line 2.\n";
    outFile << "C++ file I/O is powerful!\n";

    // — Step 4: Close the file —
    // Flushes buffered data to disk and releases the OS file handle.
    // ALWAYS close explicitly, or use block scope (RAII).
    outFile.close();

    std::cout << "Successfully wrote to hello.txt\n";
    return 0;
}
```

What appears in hello.txt :

Hello, File World!

This is line 2.

C++ file I/O is powerful!

Example 2 — Appending to a File (Log System)

```
#include <iostream>
#include <fstream>
#include <string>
#include <ctime>          // For timestamp

// Simple logger that appends entries to a log file
void writelog(const std::string& level, const std::string& message) {
    // ios::app = append mode – never erases existing content
    std::ofstream logFile("application.log", std::ios::app);

    if (!logFile.is_open()) {
        std::cerr << "CRITICAL: Cannot open log file!\n";
        return;
    }

    // Build timestamp
    std::time_t now = std::time(nullptr);
    char timeBuffer[32];
    std::strftime(timeBuffer, sizeof(timeBuffer), "%Y-%m-%d %H:%M:%S",
                  std::localtime(&now));

    // Write formatted log entry
    logFile << "[" << timeBuffer << "] [" << level << "]" << message << "\n";

    logFile.close(); // Ensure data is flushed
}

int main() {
    // Simulate an application lifecycle
    writelog("INFO", "Application started");
    writelog("INFO", "Loading configuration file");
    writelog("WARNING", "Config key 'timeout' not found, using default 30s");
    writelog("INFO", "Database connection established");
    writelog("ERROR", "Failed to load plugin: audio_engine.so");
    writelog("INFO", "Application shutting down");

    std::cout << "Log entries written to application.log\n";
    std::cout << "Run the program again – entries APPEND, not overwrite!\n";
    return 0;
}
```

application.log after second run (entries from both runs):

[2024-01-15 09:00:01] [INFO] Application started

[2024-01-15 09:00:01] [INFO] Loading configuration file

...

[2024-01-15 09:00:02] [INFO] Application started ← second run appended

Example 3 — Writing Structured Data (CSV Export)

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <iomanip>    // For setprecision

struct Product {
    int      id;
    std::string name;
    double   price;
    int      stock;
    std::string category;
};

// Export a product list to CSV format
bool exportToCSV(const std::vector<Product>& products,
                 const std::string& filename) {
    std::ofstream csv(filename);

    if (!csv.is_open()) {
        std::cerr << "ERROR: Cannot create " << filename << "\n";
        return false;
    }

    // Write CSV header row
    csv << "ID,Name,Price,Stock,Category\n";

    // Write each product as a CSV row
    for (const auto& p : products) {
        csv << p.id      << ","
            << p.name    << ","
            << std::fixed << std::setprecision(2)
            << p.price   << ","
            << p.stock   << ","
            << p.category << "\n";
    }

    csv.close();
    std::cout << "Exported " << products.size()
               << " products to " << filename << "\n";
    return true;
}
```

```
}
```

```
int main() {  
    std::vector<Product> inventory = {  
        {1001, "Wireless Mouse",    29.99, 150, "Electronics"},  
        {1002, "Mechanical Keyboard",79.99,  45, "Electronics"},  
        {1003, "USB-C Hub",        34.50,  88, "Electronics"},  
        {1004, "Desk Lamp",        24.99, 200, "Office"},  
        {1005, "Notebook (A5)",    4.99, 500, "Stationery"}  
    };  
  
    exportToCSV(inventory, "inventory_export.csv");  
  
    return 0;  
}
```

inventory_export.csv :

```
ID,Name,Price,Stock,Category  
1001,Wireless Mouse,29.99,150,Electronics  
1002,Mechanical Keyboard,79.99,45,Electronics  
1003,USB-C Hub,34.50,88,Electronics  
1004,Desk Lamp,24.99,200,Office  
1005,Notebook (A5),4.99,500,Stationery
```

ifstream — Reading from Files

ifstream (Input File Stream) reads data from existing files.

Example 4 — Reading a File Line by Line

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
    // Open file for reading
    // If the file does NOT exist, is_open() returns false
    std::ifstream inFile("hello.txt");

    if (!inFile.is_open()) {
        std::cerr << "ERROR: hello.txt not found.\n";
        std::cerr << "Make sure you ran Example 1 first to create it.\n";
        return 1;
    }

    std::string line;          // Buffer to hold each line
    int lineNumber = 0;

    std::cout << "Contents of hello.txt:\n";
    std::cout << std::string(40, '-') << "\n";

    // getline(stream, string) reads one complete line per call.
    // The while loop runs until getline() returns false (end of file or error).
    // getline() STRIPS the '\n' character – it's not included in 'line'.
    while (std::getline(inFile, line)) {
        lineNumber++;
        std::cout << std::setw(3) << lineNumber << " | " << line << "\n";
    }

    std::cout << std::string(40, '-') << "\n";
    std::cout << "Total lines read: " << lineNumber << "\n";

    inFile.close();
    return 0;
}
```

Example 5 — Reading a CSV File into Structs

```
#include <iostream>
#include <fstream>
#include <sstream>    // stringstream for parsing CSV tokens
#include <string>
#include <vector>

struct Product {
    int      id;
    std::string name;
    double   price;
    int      stock;
    std::string category;
};

// Parse a CSV line into a Product struct
// CSV format: id,name,price,stock,category
Product parseCSVLine(const std::string& line) {
    Product p;
    std::stringstream ss(line);
    std::string token;

    // Each getline(ss, token, ',') reads up to the next comma
    std::getline(ss, token, ','); p.id      = std::stoi(token);
    std::getline(ss, p.name, ',');           // Name (may contain spaces)
    std::getline(ss, token, ','); p.price   = std::stod(token);
    std::getline(ss, token, ','); p.stock   = std::stoi(token);
    std::getline(ss, p.category);           // Last field, no comma

    return p;
}

int main() {
    std::ifstream csv("inventory_export.csv");

    if (!csv.is_open()) {
        std::cerr << "ERROR: inventory_export.csv not found.\n";
        std::cerr << "Run Example 3 first to generate it.\n";
        return 1;
    }

    std::vector<Product> products;
```

```

std::string line;

// Skip the header row
std::getline(csv, line);
std::cout << "Skipped header: " << line << "\n\n";

// Read each data row
while (std::getline(csv, line)) {
    if (!line.empty()) {
        products.push_back(parseCSVLine(line));
    }
}
csv.close();

// Display the loaded data
std::cout << "Loaded " << products.size() << " products:\n";
std::cout << std::string(65, '=') << "\n";
std::cout << std::left
    << std::setw(6) << "ID"
    << std::setw(25) << "Name"
    << std::setw(10) << "Price"
    << std::setw(8) << "Stock"
    << "Category\n";
std::cout << std::string(65, '-') << "\n";

for (const auto& p : products) {
    std::cout << std::left
        << std::setw(6) << p.id
        << std::setw(25) << p.name
        << "$" << std::setw(9) << std::fixed
        << std::setprecision(2) << p.price
        << std::setw(8) << p.stock
        << p.category << "\n";
}

return 0;
}

```

Example 6 — Word Frequency Counter

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <map>
#include <algorithm>
#include <cctype>

// Remove punctuation and convert to lowercase
std::string cleanWord(std::string word) {
    // Remove non-alpha characters from both ends
    word.erase(std::remove_if(word.begin(), word.end(),
        [](char c) { return !std::isalpha(c); }), word.end());
    // Convert to lowercase
    std::transform(word.begin(), word.end(), word.begin(),
        [](char c) { return std::tolower(c); });
    return word;
}

int main() {
    // First, create a sample text file to analyze
    {
        std::ofstream sample("sample_text.txt");
        sample << "The quick brown fox jumps over the lazy dog.\n";
        sample << "The dog barked at the fox.\n";
        sample << "A fox is a clever animal. The fox ran away.\n";
    }

    std::ifstream textFile("sample_text.txt");
    if (!textFile.is_open()) {
        std::cerr << "Cannot open sample_text.txt\n";
        return 1;
    }

    // Count word frequencies using a map
    std::map<std::string, int> wordCount;
    std::string line;

    while (std::getline(textFile, line)) {
        std::istringstream iss(line);
        std::string word;
```

```

    while (iss >> word) {
        std::string clean = cleanWord(word);
        if (!clean.empty()) {
            wordCount[clean]++; // Increment count for this word
        }
    }
}

textFile.close();

// Sort by frequency (descending) using a vector of pairs
std::vector<std::pair<std::string, int>> sorted(
    wordCount.begin(), wordCount.end());
std::sort(sorted.begin(), sorted.end(),
    [](const auto& a, const auto& b) { return a.second > b.second; });

// Display results
std::cout << "Word Frequency Analysis:\n";
std::cout << std::string(30, '=') << "\n";
std::cout << std::left << std::setw(20) << "Word"
    << "Count\n";
std::cout << std::string(30, '-') << "\n";
for (const auto& [word, count] : sorted) {
    std::cout << std::left << std::setw(20) << word << count << "\n";
}

return 0;
}

```

fstream — Reading AND Writing

`fstream` combines both directions. You must specify open mode flags explicitly.

Example 7 — In-Place File Update

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

int main() {
    const std::string filename = "config.txt";

    // — Step 1: Create initial config file —
    {
        std::ofstream create(filename);
        create << "# Application Configuration\n";
        create << "debug_mode=false\n";
        create << "max_connections=10\n";
        create << "server_host=localhost\n";
        create << "log_level=info\n";
    }
    std::cout << "Created config.txt\n";

    // — Step 2: Read all lines into memory —
    std::vector<std::string> lines;
    {
        std::fstream file(filename, std::ios::in);
        std::string line;
        while (std::getline(file, line)) {
            lines.push_back(line);
        }
    }
    std::cout << "Read " << lines.size() << " lines\n";

    // — Step 3: Modify specific settings in memory —
    for (auto& line : lines) {
        if (line.find("debug_mode=") == 0) {
            line = "debug_mode=true";           // Enable debug mode
        }
        if (line.find("max_connections=") == 0) {
            line = "max_connections=50";        // Increase limit
        }
    }

    // — Step 4: Write modified content back —
```



```

{
    // ios::out | ios::trunc = write mode, erase old content first
    std::fstream file(filename, std::ios::out | std::ios::trunc);
    for (const auto& line : lines) {
        file << line << "\n";
    }
}

std::cout << "Config updated.\n\n";

// — Step 5: Verify changes —
std::fstream verify(filename, std::ios::in);
std::cout << "Updated config.txt:\n";
std::cout << std::string(30, '-') << "\n";
std::string line;
while (std::getline(verify, line)) {
    std::cout << line << "\n";
}

return 0;
}

```

Example 8 — Binary File Read/Write with fstream

```
#include <iostream>
#include <fstream>
#include <cstring>

// Fixed-size config struct for binary storage
// (All fields MUST have fixed sizes – no std::string!)
struct AppConfig {
    char    appName[32];    // Fixed char array
    int     version;
    float   masterVolume;
    bool    fullscreen;
    int     resolutionW;
    int     resolutionH;
    char    language[8];
};

int main() {
    const char* configPath = "settings.bin";

    // — Write binary config —
    {
        AppConfig cfg;
        std::strncpy(cfg.appName, "GameEngine", sizeof(cfg.appName));
        std::strncpy(cfg.language, "en", sizeof(cfg.language));
        cfg.version      = 3;
        cfg.masterVolume = 0.75f;
        cfg.fullscreen   = false;
        cfg.resolutionW  = 1920;
        cfg.resolutionH  = 1080;

        // ios::binary prevents \n → \r\n translation on Windows
        std::fstream file(configPath, std::ios::out | std::ios::binary);
        if (!file) { std::cerr << "Cannot write config\n"; return 1; }

        // write(pointer_to_data, byte_count)
        // reinterpret_cast converts struct pointer to char pointer
        file.write(reinterpret_cast<const char*>(&cfg), sizeof(AppConfig));
        std::cout << "Binary config written (" << sizeof(AppConfig) << " bytes)\n";
    }

    // — Read binary config back —
```

```

{
    AppConfig cfg;
    std::fstream file(configPath, std::ios::in | std::ios::binary);
    if (!file) { std::cerr << "Cannot read config\n"; return 1; }

    file.read(reinterpret_cast<char*>(&cfg), sizeof(AppConfig));

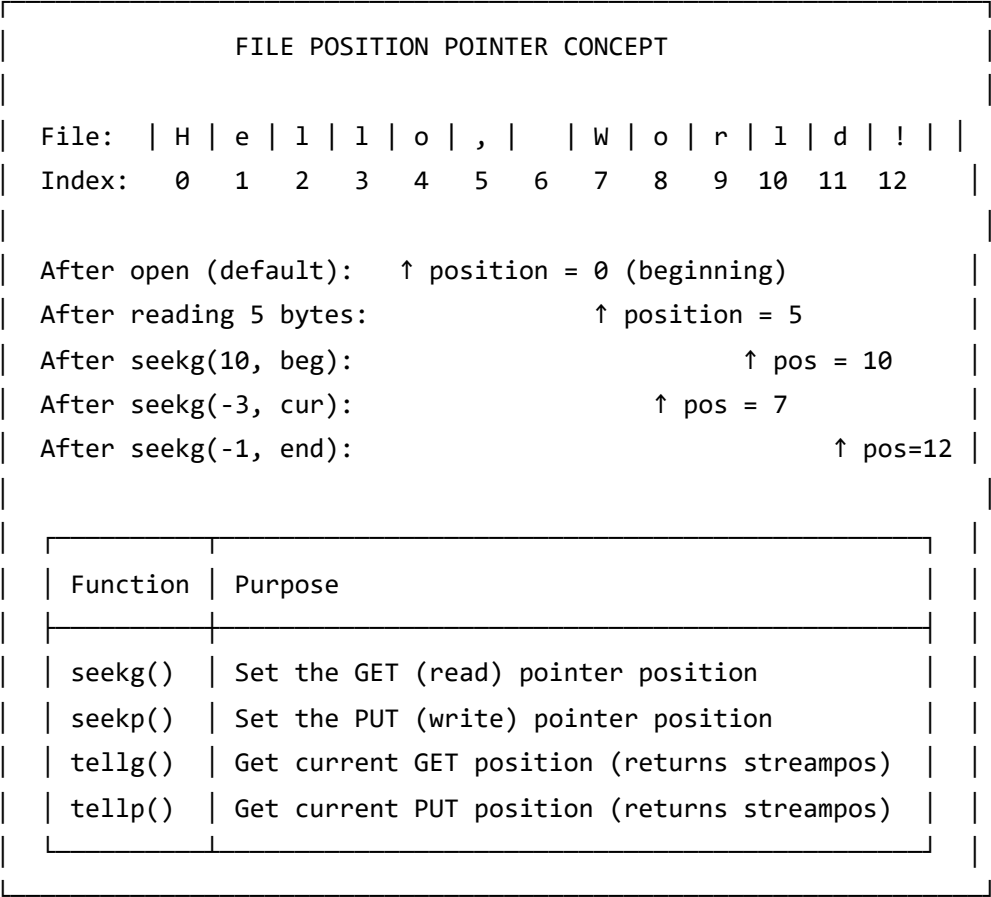
    std::cout << "\nLoaded configuration:\n";
    std::cout << "  App Name   : " << cfg.appName      << "\n";
    std::cout << "  Version    : " << cfg.version         << "\n";
    std::cout << "  Volume     : " << cfg.masterVolume    << "\n";
    std::cout << "  Fullscreen : " << (cfg.fullscreen ? "Yes" : "No") << "\n";
    std::cout << "  Resolution : " << cfg.resolutionW
                << "x" << cfg.resolutionH << "\n";
    std::cout << "  Language   : " << cfg.language        << "\n";
}

return 0;
}

```

18.3 File Position Pointers

Concept Diagram



Seek Origin Constants

Constant	Meaning
ios::beg	Offset from the beginning of the file
ios::cur	Offset from the current position
ios::end	Offset from the end of the file (usually negative)

Example 9 — Exploring seekg and tellg

```
#include <iostream>
#include <fstream>
#include <iomanip>

int main() {
    // Create test file: 26 uppercase letters
    {
        std::ofstream create("alphabet.txt");
        create << "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
    }

    std::ifstream file("alphabet.txt");
    if (!file) { std::cerr << "Cannot open alphabet.txt\n"; return 1; }

    char ch;

    // — Get file size —
    file.seekg(0, std::ios::end);          // Jump to end
    std::streampos fileSize = file.tellg(); // Position at end = file size
    std::cout << "File size: " << fileSize << " bytes\n\n";

    // — Seek to beginning —
    file.seekg(0, std::ios::beg);
    std::cout << "Position after rewind: " << file.tellg() << "\n";

    // — Read character by character from position 0 —
    std::cout << "\nReading first 5 characters:\n";
    for (int i = 0; i < 5; i++) {
        file.get(ch);
        std::cout << " pos=" << file.tellg() << " char='" << ch << "'\n";
    }

    // — Absolute seek from beginning —
    file.seekg(10, std::ios::beg); // Jump to byte 10 (0-indexed)
    file.get(ch);
    std::cout << "\nChar at absolute position 10: '" << ch << "' (expect 'K')\n";

    // — Relative seek from current position —
    file.seekg(4, std::ios::cur); // Skip 4 bytes forward
    file.get(ch);
    std::cout << "After skipping 4 from pos 11: '" << ch << "' (expect 'P')\n";
```

```
// — Seek from end —  
file.seekg(-3, std::ios::end);    // 3 bytes before end  
file.get(ch);  
std::cout << "3rd character from end:      '" << ch << "' (expect 'X')\n";  
  
file.close();  
return 0;  
}
```

Example 10 — Random Access Binary Records

```
#include <iostream>
#include <fstream>
#include <cstring>
#include <iomanip>

// Fixed-size record — CRITICAL for random access
// Every record must have the same byte size
struct EmployeeRecord {
    int    id;
    char   name[40];    // Fixed size — NOT std::string
    double salary;
    bool   isActive;
};

void printRecord(const EmployeeRecord& e, int index) {
    std::cout << "  [" << index << "] ID:" << std::setw(5) << e.id
               << "  Name: " << std::left << std::setw(20) << e.name
               << std::right
               << "  Salary: $" << std::fixed << std::setprecision(2)
               << e.salary
               << "  Status: " << (e.isActive ? "Active" : "Inactive")
               << "\n";
}

int main() {
    const char* dbFile = "employees.bin";

    // — STEP 1: Create the binary record file —
    {
        std::fstream file(dbFile, std::ios::out | std::ios::binary);

        EmployeeRecord records[] = {
            {1001, "Alice Johnson", 75000.0, true },
            {1002, "Bob Smith",     62000.0, true },
            {1003, "Carol Williams", 88000.0, true },
            {1004, "David Brown",   55000.0, false},
            {1005, "Eva Martinez",  95000.0, true }
        };

        for (const auto& r : records) {
            file.write(reinterpret_cast<const char*>(&r), sizeof(EmployeeRecord));
        }
    }
}
```

```

    }
    std::cout << "Created database with "
                << sizeof(records)/sizeof(records[0]) << " records.\n";
}

// — STEP 2: Read all records (sequential access) —
{
    std::fstream file(dbFile, std::ios::in | std::ios::binary);
    EmployeeRecord emp;
    int index = 0;

    std::cout << "\nAll employees (sequential read):\n";
    std::cout << std::string(70, '=') << "\n";
    while (file.read(reinterpret_cast<char*>(&emp), sizeof(EmployeeRecord))) {
        printRecord(emp, index++);
    }
}

// — STEP 3: Random access – read record at index 2 (Carol) —
{
    std::fstream file(dbFile, std::ios::in | std::ios::binary);
    int targetIndex = 2; // Carol is at index 2

    // Calculate byte offset: index × record_size
    std::streamoff offset = static_cast<std::streamoff>(targetIndex)
                           * sizeof(EmployeeRecord);

    file.seekg(offset, std::ios::beg);

    EmployeeRecord emp;
    file.read(reinterpret_cast<char*>(&emp), sizeof(EmployeeRecord));

    std::cout << "\nRandom access – record at index " << targetIndex << ":\n";
    printRecord(emp, targetIndex);
}

// — STEP 4: Update a record (Bob's salary) —
{
    std::fstream file(dbFile,
                      std::ios::in | std::ios::out | std::ios::binary);
    int targetIndex = 1; // Bob is at index 1
    std::streamoff offset = static_cast<std::streamoff>(targetIndex)
                           * sizeof(EmployeeRecord);

```



```

    // Read current record
    EmployeeRecord emp;
    file.seekg(offset, std::ios::beg);
    file.read(reinterpret_cast<char*>(&emp), sizeof(EmployeeRecord));
    std::cout << "\nBefore update: ";
    printRecord(emp, targetIndex);

    // Modify in memory
    emp.salary = 68000.0;

    // Write back at the same offset
    file.seekp(offset, std::ios::beg);
    file.write(reinterpret_cast<const char*>(&emp), sizeof(EmployeeRecord));
    std::cout << "After update: ";
    printRecord(emp, targetIndex);
}

// — STEP 5: Verify the update —
{
    std::fstream file(dbFile, std::ios::in | std::ios::binary);
    int targetIndex = 1;
    std::streamoff offset = static_cast<std::streamoff>(targetIndex)
                           * sizeof(EmployeeRecord);
    file.seekg(offset, std::ios::beg);
    EmployeeRecord emp;
    file.read(reinterpret_cast<char*>(&emp), sizeof(EmployeeRecord));
    std::cout << "\nVerification (re-read Bob's record):\n";
    printRecord(emp, targetIndex);
}

return 0;
}

```

File State Flags

```
#include <iostream>
#include <fstream>

int main() {
    std::ifstream f("hello.txt");

    // State checking functions
    std::cout << "good(): " << f.good() // true = no errors
               << " eof(): " << f.eof() // true = end of file reached
               << " fail(): " << f.fail() // true = logical error
               << " bad(): " << f.bad() // true = hardware I/O error
               << "\n";

    std::string word;
    while (f >> word) { /* read... */ }

    // After reading past end:
    if (f.eof()) std::cout << "Reached end of file normally.\n";
    if (f.fail()) std::cout << "A read error occurred.\n";

    // Reset error flags to reuse the stream
    f.clear();
    f.seekg(0, std::ios::beg); // Rewind
    std::cout << "Stream reset – ready to re-read.\n";

    return 0;
}
```

Common Mistakes — File I/O

Mistake	Consequence	Correct Approach
Not checking <code>is_open()</code>	Silent failure / crash	Always check after opening
Using <code>>></code> to read full lines	Stops at first space	Use <code>std::getline()</code> for lines
Forgetting <code>close()</code>	Buffered data lost; resource leak	Use RAII (block scope) or explicit <code>close()</code>

Mistake	Consequence	Correct Approach
Using text mode for raw binary data	Data corruption on Windows	Add <code>ios::binary</code> flag
Not calling <code>clear()</code> before <code>seekg()</code> after EOF	Seek silently fails	Clear flags before re-seeking
Hardcoding paths with <code>\</code>	Fails on Linux/macOS	Use <code>/</code> or <code>std::filesystem::path</code>

💡 Pro Tips — File I/O

Tip 1 — RAII File Management: Use block scope instead of manual `close()` :

```
{
    std::ofstream f("file.txt");
    f << "data\n";
} // f.close() called automatically here – even if exception thrown
```

Tip 2 — C++17 `std::filesystem` : Check file existence safely:

```
#include <filesystem>
if (std::filesystem::exists("data.txt")) { /* safe to open */ }
```

Tip 3 — Large Files: Never load an entire large file into memory. Process it line-by-line in a loop.

Tip 4 — Atomic Writes: Write to a temp file then rename for crash-safe file updates:

```
std::ofstream tmp("data.tmp");
// ... write ...
tmp.close();
std::filesystem::rename("data.tmp", "data.txt"); // Atomic on most systems
```

Quick Reference — File I/O

```
// Open modes
std::ofstream out("f.txt");           // Write (create/truncate)
std::ofstream app("f.txt", std::ios::app); // Append
std::ifstream in("f.txt");           // Read
std::fstream rw("f.txt", std::ios::in|std::ios::out); // Both

// Write
out << "text\n";                     // Formatted write
out.write(buf, count);                // Raw binary write

// Read
std::getline(in, line);               // Full line (no '\n')
in >> word;                          // Word/token (stops at space)
in.read(buf, count);                 // Raw binary read

// Position
in.seekg(pos, std::ios::beg);        // Move read pointer
in.tellg();                          // Get read position
out.seekp(pos, std::ios::beg);       // Move write pointer
out.tellp();                         // Get write position

// State
f.good() f.eof() f.fail() f.bad()
f.clear()           // Reset error flags
f.is_open()        // Check if open
f.close()          // Close
```

Exercise — File I/O

Problem: Build a simple address book program that saves contacts to `contacts.txt`, loads them back on startup, supports searching by name, and can delete a contact by name.

Solution:

```

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <algorithm>

struct Contact {
    std::string name;
    std::string phone;
    std::string email;
};

const std::string FILE_PATH = "contacts.txt";

// — Save all contacts to file —
void saveAll(const std::vector<Contact>& contacts) {
    std::ofstream out(FILE_PATH);
    if (!out) { std::cerr << "Cannot save contacts!\n"; return; }

    for (const auto& c : contacts) {
        // Pipe-delimited format: name|phone|email
        out << c.name << "|" << c.phone << "|" << c.email << "\n";
    }
    std::cout << "Saved " << contacts.size() << " contacts.\n";
}

// — Load contacts from file —
std::vector<Contact> loadAll() {
    std::vector<Contact> contacts;
    std::ifstream in(FILE_PATH);
    if (!in) return contacts; // File doesn't exist yet – return empty

    std::string line;
    while (std::getline(in, line)) {
        if (line.empty()) continue;
        std::stringstream ss(line);
        Contact c;
        std::getline(ss, c.name, '|');
        std::getline(ss, c.phone, '|');
        std::getline(ss, c.email);
        contacts.push_back(c);
    }
}

```

```

    return contacts;
}

// — Search by name (case-insensitive partial match) —
void search(const std::vector<Contact>& contacts, const std::string& query) {
    std::string lq = query;
    std::transform(lq.begin(), lq.end(), lq.begin(), ::tolower);

    bool found = false;
    for (const auto& c : contacts) {
        std::string ln = c.name;
        std::transform(ln.begin(), ln.end(), ln.begin(), ::tolower);
        if (ln.find(lq) != std::string::npos) {
            std::cout << "  Name : " << c.name << "\n"
                      << "  Phone: " << c.phone << "\n"
                      << "  Email: " << c.email << "\n\n";
            found = true;
        }
    }
    if (!found) std::cout << "  No contacts found for '" << query << "'\n";
}

// — Delete by exact name —
bool deleteByName(std::vector<Contact>& contacts, const std::string& name) {
    auto before = contacts.size();
    contacts.erase(
        std::remove_if(contacts.begin(), contacts.end(),
            [&name](const Contact& c) { return c.name == name; }),
        contacts.end()
    );
    return contacts.size() < before;
}

int main() {
    // Load existing contacts
    auto contacts = loadAll();
    std::cout << "Loaded " << contacts.size() << " existing contacts.\n\n";

    // Add new contacts
    contacts.push_back({"Alice Johnson", "555-0101", "alice@example.com"});
    contacts.push_back({"Bob Smith", "555-0202", "bob@work.com"});
    contacts.push_back({"Carol Williams", "555-0303", "carol@personal.net"});
    contacts.push_back({"Alice Wang", "555-0404", "a.wang@tech.org"});
}

```

```

saveAll(contacts);

// Search
std::cout << "\n--- Search 'alice' ---\n";
search(contacts, "alice");

std::cout << "--- Search 'Smith' ---\n";
search(contacts, "Smith");

// Delete
std::cout << "--- Deleting 'Bob Smith' ---\n";
if (deleteByName(contacts, "Bob Smith")) {
    std::cout << " Deleted successfully.\n";
    saveAll(contacts);
} else {
    std::cout << " Contact not found.\n";
}

// Show remaining
std::cout << "\n--- Remaining Contacts ---\n";
for (const auto& c : contacts) {
    std::cout << " " << c.name << " | " << c.phone << "\n";
}

return 0;
}

```

Section 19: Exception Handling

19.1 Basics of Exception Handling

Learning Objectives

- Understand why exceptions exist and the problems they solve
- Learn how exception propagation and stack unwinding work
- Distinguish exception handling from returning error codes
- Know when to use exceptions vs. other error strategies

- Understand exception safety levels

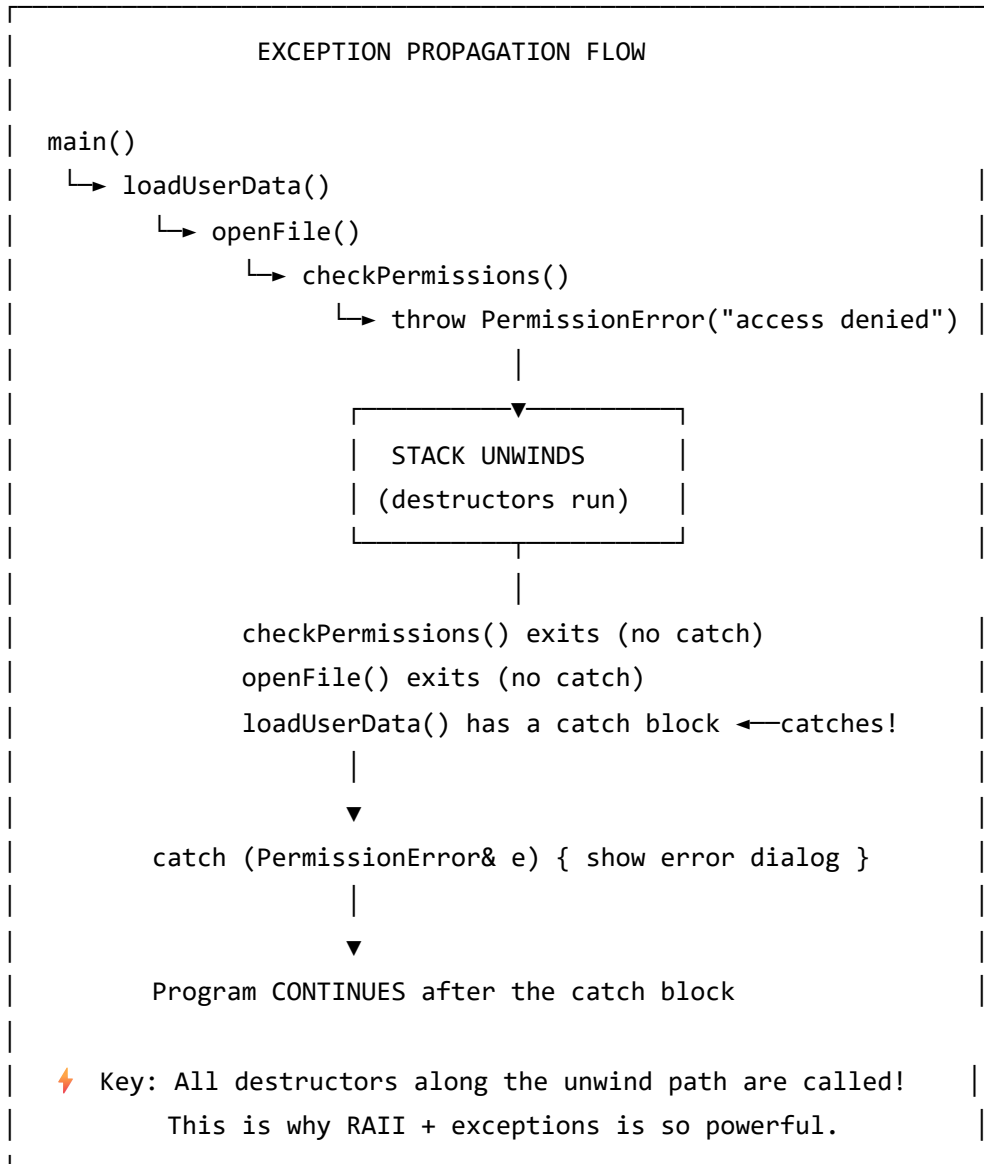
Introduction

Imagine writing a function that opens a file, reads a number from it, divides by that number, and writes the result to another file. At every step, something can go wrong: the file might not exist, the "number" might be the word "hello", the number might be zero, or the output disk might be full. In traditional C, you'd handle each case by checking a return value — and forgetting to check even one means a silent bug.

Exception handling is C++'s built-in mechanism for separating the normal execution path from error handling. When something goes wrong, a function **throws** an exception object. This exception travels back up the call stack — skipping all intermediate code — until a **catch** block handles it. If nobody handles it, the program terminates.

The key insight: exceptions make errors *impossible to silently ignore*. A function that throws and is never caught terminates the program — forcing developers to actually deal with errors. Combined with RAII (destructors that auto-release resources), exceptions allow very clean code that handles errors correctly even in complex scenarios.

Exception Propagation — Flow Diagram



Exceptions vs. Error Codes — Comparison

EXCEPTIONS vs RETURN ERROR CODES	
ERROR CODES	EXCEPTIONS
Can be silently ignored by callers	Cannot be ignored (unhandled = program terminates)
Clutter every call site with if/else checks	Happy-path code stays clean
Error context lost after return	Exception object carries full context (message, type, data)
Better for frequent, expected conditions	Better for exceptional, rare error conditions
Better in inner loops (no stack overhead)	Better for constructor errors (constructors can't return a value)

19.2 try, catch, and throw

Syntax Diagram

try / catch / throw – FULL SYNTAX

```
try {  
    // ① Normal code that might throw  
    doRiskyWork();  
    throw SomeException("what went wrong"); // ② throw  
}  
catch (const MostSpecificException& e) {    // ③ first  
    // Handle most specific type first  
}  
catch (const LessSpecificException& e) {    // ④ more  
    // Handle parent types after children  
}  
catch (const std::exception& e) {           // ⑤ std base  
    // Catch any std:: exception not caught above  
}  
catch (...) {                               // ⑥ catch-all  
    // Catch ANYTHING (even non-exception types)  
}  
// ⑦ Execution continues here after catch handles it
```

Example 1 — Basic try / catch / throw

```
#include <iostream>
#include <stdexcept> // Standard exception classes
#include <string>

// Function that performs safe division – throws on zero denominator
double safeDivide(double numerator, double denominator) {
    if (denominator == 0.0) {
        // throw creates an exception object and begins unwinding
        throw std::invalid_argument(
            "Division by zero: denominator cannot be zero"
        );
    }
    return numerator / denominator;
}

int main() {
    // — Case 1: Normal division (no exception) —
    std::cout << "Case 1: Normal division\n";
    try {
        double result = safeDivide(10.0, 4.0);
        std::cout << " 10 / 4 = " << result << "\n"; // 2.5
        std::cout << " (No exception thrown)\n";
    }
    catch (const std::invalid_argument& e) {
        // NOT executed – there was no exception
        std::cerr << " Error: " << e.what() << "\n";
    }

    // — Case 2: Division by zero (exception thrown) —
    std::cout << "\nCase 2: Division by zero\n";
    try {
        double result = safeDivide(10.0, 0.0); // ← throws here
        // Everything below the throw is SKIPPED – execution jumps to catch
        std::cout << " Result: " << result << "\n"; // Never executes
        std::cout << " This line also never executes\n";
    }
    catch (const std::invalid_argument& e) {
        // e.what() returns the message passed to the exception constructor
        std::cerr << " Caught: " << e.what() << "\n";
    }
}
```

```
std::cout << "\nProgram continues normally after the catch block.\n";  
return 0;  
}
```

Output:

Case 1: Normal division

10 / 4 = 2.5

(No exception thrown)

Case 2: Division by zero

Caught: Division by zero: denominator cannot be zero

Program continues normally after the catch block.

Example 2 — Multiple catch Blocks

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

// Function that throws different exceptions based on input
std::string getUserData(int userId) {
    if (userId < 0) {
        throw std::invalid_argument(
            "User ID must be non-negative, got: " + std::to_string(userId)
        );
    }
    if (userId == 0) {
        throw std::domain_error(
            "User ID 0 is reserved for the system"
        );
    }
    if (userId > 9999) {
        throw std::out_of_range(
            "User ID exceeds maximum (9999), got: " + std::to_string(userId)
        );
    }
    // Simulate users
    if (userId == 42) return "Alice Johnson";
    return "Unknown User";
}

int main() {
    std::vector<int> testIds = {42, -5, 0, 99999, 100};

    for (int id : testIds) {
        std::cout << "Looking up user ID " << id << "... ";
        try {
            std::string name = getUserData(id);
            std::cout << "Found: " << name << "\n";
        }
        // — Multiple catch blocks — checked IN ORDER, most specific FIRST —
        catch (const std::invalid_argument& e) {
            // Caught if type matches exactly or derives from invalid_argument
            std::cerr << "[INVALID ARG] " << e.what() << "\n";
        }
    }
}
```

```

catch (const std::domain_error& e) {
    std::cerr << "[DOMAIN ERROR] " << e.what() << "\n";
}
catch (const std::out_of_range& e) {
    std::cerr << "[OUT OF RANGE] " << e.what() << "\n";
}
catch (const std::logic_error& e) {
    // Parent class of the three above – catches any unmatched logic errors
    std::cerr << "[LOGIC ERROR] " << e.what() << "\n";
}
catch (const std::exception& e) {
    // Catches ANY std:: exception not matched above
    std::cerr << "[EXCEPTION] " << e.what() << "\n";
}
catch (...) {
    // Catches ANYTHING – even non-exception types like throw 42;
    std::cerr << "[UNKNOWN ERROR]\n";
}
}
return 0;
}

```

Example 3 — Custom Exception Classes

```
#include <iostream>
#include <stdexcept>
#include <string>

// — Base custom exception —
// Always inherit from std::exception (or a subclass)
class AppException : public std::exception {
protected:
    std::string message_;
    int         errorCode_;

public:
    AppException(const std::string& msg, int code)
        : message_(msg), errorCode_(code) {}

    // Override what() — must be noexcept per the standard
    const char* what() const noexcept override {
        return message_.c_str();
    }

    int code() const noexcept { return errorCode_; }
};

// — Specific exception: database error —
class DatabaseException : public AppException {
    std::string failedQuery_;

public:
    DatabaseException(const std::string& msg, const std::string& query)
        : AppException("[DB-ERROR] " + msg, 500), failedQuery_(query) {}

    const std::string& query() const { return failedQuery_; }
};

// — Specific exception: authentication error —
class AuthException : public AppException {
    std::string username_;

public:
    AuthException(const std::string& msg, const std::string& user)
        : AppException("[AUTH-ERROR] " + msg, 401), username_(user) {}
};
```



```

    const std::string& username() const { return username_; }
};

// — Functions that throw custom exceptions —
void connectDatabase(const std::string& host) {
    if (host.empty()) {
        throw DatabaseException("Host cannot be empty", "CONNECT");
    }
    if (host == "bad-host") {
        throw DatabaseException("Connection refused", "CONNECT " + host);
    }
    std::cout << "  DB connected: " << host << "\n";
}

void authenticate(const std::string& user, const std::string& pass) {
    if (user.empty()) {
        throw AuthException("Username is required", user);
    }
    if (pass != "correct_password") {
        throw AuthException("Wrong password for user: " + user, user);
    }
    std::cout << "  Authenticated: " << user << "\n";
}

int main() {
    struct Test { std::string host, user, pass, label; };
    std::vector<Test> tests = {
        {"db.example.com", "admin", "correct_password", "Happy path"},
        {"", "admin", "correct_password", "Empty host"},
        {"bad-host", "admin", "correct_password", "Bad host"},
        {"db.example.com", "admin", "wrong_pass", "Wrong password"},
        {"db.example.com", "", "correct_password", "Empty username"},
    };

    for (const auto& t : tests) {
        std::cout << "\n--- " << t.label << " ---\n";
        try {
            connectDatabase(t.host);
            authenticate(t.user, t.pass);
            std::cout << "  Login successful!\n";
        }
        catch (const DatabaseException& e) {

```

```

        // Note: we can access extra info from custom exception
        std::cerr << " " << e.what()
                << " (code=" << e.code() << ")\n";
        std::cerr << " Failed query: " << e.query() << "\n";
    }
    catch (const AuthException& e) {
        std::cerr << " " << e.what()
                << " (code=" << e.code() << ")\n";
        std::cerr << " User: " << e.username() << "\n";
    }
    catch (const AppException& e) {
        // Catch any other app exceptions
        std::cerr << " App error: " << e.what() << "\n";
    }
}
return 0;
}

```

Example 4 — Re-throwing and Stack Unwinding with RAII

```
#include <iostream>
#include <stdexcept>
#include <string>

// RAII resource – shows destructor called during unwinding
class ManagedResource {
    std::string label_;
public:
    ManagedResource(const std::string& label) : label_(label) {
        std::cout << "    [ACQUIRE] " << label_ << "\n";
    }
    ~ManagedResource() {
        // This ALWAYS runs – even during exception unwinding
        std::cout << "    [RELEASE] " << label_ << "\n";
    }
};

void deepFunction() {
    ManagedResource r("Network Socket");
    std::cout << "    [deepFunction] About to throw...\n";
    throw std::runtime_error("Something failed deep in the stack");
    // r's destructor runs even though we never reach the closing brace
}

void middleFunction() {
    ManagedResource r("File Handle");
    std::cout << "    [middleFunction] Calling deepFunction...\n";
    try {
        deepFunction();
    }
    catch (const std::runtime_error& e) {
        std::cerr << "    [middleFunction] Caught, logging: " << e.what() << "\n";
        throw; // ← Re-throw the SAME exception (bare throw preserves type)
    }
    // r's destructor runs here (scope exit, whether normal or exception)
}

void outerFunction() {
    ManagedResource r("Database Connection");
    std::cout << "[outerFunction] Calling middleFunction...\n";
    try {
```

```

        middleFunction();
    }
    catch (const std::exception& e) {
        std::cerr << "[outerFunction] Final handler: " << e.what() << "\n";
        // Exception fully handled here – execution continues normally
    }
}

int main() {
    std::cout << "=== PROGRAM START ===\n";
    outerFunction();
    std::cout << "=== PROGRAM END (ran successfully) ===\n";
    return 0;
}

```

Output (notice destructor order — LIFO):

```

=== PROGRAM START ===
[outerFunction] Calling middleFunction...
    [ACQUIRE] Database Connection
[middleFunction] Calling deepFunction...
    [ACQUIRE] File Handle
    [ACQUIRE] Network Socket
    [deepFunction] About to throw...
    [RELEASE] Network Socket      ← unwinding: innermost first
[middleFunction] Caught, logging: Something failed deep in the stack
    [RELEASE] File Handle        ← middleFunction's resource freed
[outerFunction] Final handler: Something failed deep in the stack
    [RELEASE] Database Connection ← outerFunction's resource freed
=== PROGRAM END (ran successfully) ===

```

Example 5 — Exception Safety: Strong Guarantee

```
#include <iostream>
#include <stdexcept>
#include <vector>
#include <string>

/*
 * Exception Safety Levels:
 * 1. No-throw:   Function never throws (noexcept)
 * 2. Strong:     If throws, object state is completely unchanged
 * 3. Basic:      If throws, object is in a valid (but possibly modified) state
 * 4. None:       If throws, object may be in an invalid/unknown state
 */
class SafeStack {
    std::vector<int> data_;
    size_t          maxSize_;

public:
    explicit SafeStack(size_t maxSize) : maxSize_(maxSize) {}

    // STRONG GUARANTEE: copy-and-swap idiom
    // Either fully succeeds, or data_ is completely unchanged.
    void push(int value) {
        if (data_.size() >= maxSize_) {
            throw std::overflow_error(
                "Stack is full (max=" + std::to_string(maxSize_) + ")");
        };
        // ↑ Exception thrown BEFORE any modification – original safe

        // Make a temporary copy, modify it, then swap atomically
        std::vector<int> temp = data_;    // This might throw (memory), but
                                           // data_ is untouched if it does
        temp.push_back(value);           // Modify the copy
        data_ = std::move(temp);         // No-throw swap (commit)

        std::cout << "  Pushed " << value
                   << " (size=" << data_.size() << ")\n";
    }

    // NO-THROW GUARANTEE: marked noexcept
    int top() const noexcept {
```

```

        return data_.empty() ? -1 : data_.back();
    }

// STRONG GUARANTEE
void pop() {
    if (data_.empty()) {
        throw std::underflow_error("Cannot pop from empty stack");
    }
    data_.pop_back(); // No-throw if value_type is int
}

size_t size() const noexcept { return data_.size(); }
bool empty() const noexcept { return data_.empty(); }

void print() const {
    std::cout << " Stack [" << data_.size() << "]: ";
    for (int v : data_) std::cout << v << " ";
    std::cout << "\n";
}
};

int main() {
    SafeStack stack(3);

    try {
        stack.push(10);
        stack.push(20);
        stack.push(30);
        stack.print();

        // Attempt to push to a full stack
        stack.push(40); // Throws overflow_error
    }
    catch (const std::overflow_error& e) {
        std::cerr << " Caught: " << e.what() << "\n";
    }

    // Verify strong guarantee – stack unchanged after exception
    std::cout << " After overflow attempt:\n";
    stack.print(); // Still [10, 20, 30]

    // Pop operations
    try {

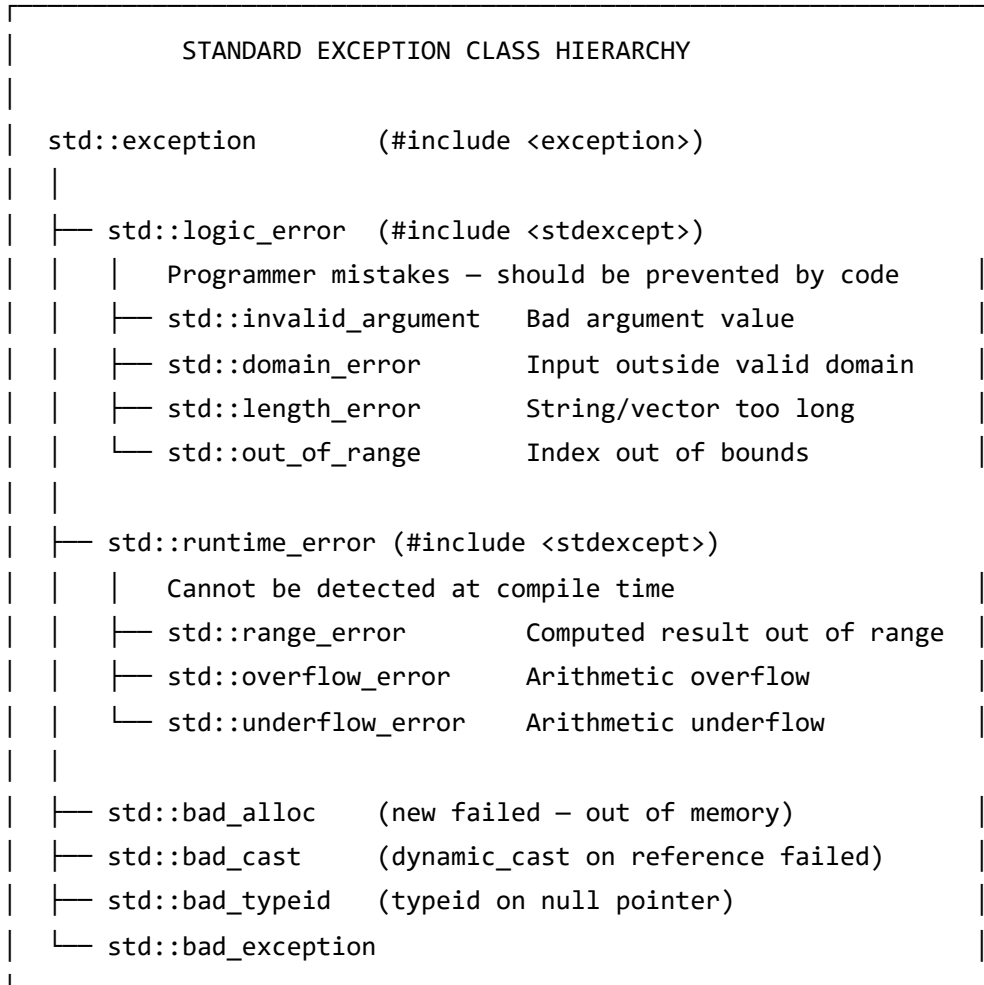
```

```
    while (!stack.empty()) {
        std::cout << " Popped: " << stack.top() << "\n";
        stack.pop();
    }
    stack.pop(); // Throws underflow_error
}
catch (const std::underflow_error& e) {
    std::cerr << " Caught: " << e.what() << "\n";
}

return 0;
}
```

19.3 Standard Exceptions

Hierarchy Diagram



Example 6 — Standard Exceptions in Action

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
#include <new>          // std::bad_alloc

// Helper macro to run a test case
#define RUN_TEST(label, code) \
    std::cout << "\n[" << (label) << "]\n"; \
    try { code } \
    catch (const std::exception& e) { \
        std::cerr << "  CAUGHT (" << typeid(e).name() << "): " \
            << e.what() << "\n"; \
    }

int main() {
    // — std::out_of_range —
    RUN_TEST("out_of_range from vector::at()",
        std::vector<int> v = {1, 2, 3};
        int x = v.at(10);    // .at() bounds-checks; .operator[] does not
        std::cout << x << "\n";
    )

    // — std::out_of_range from std::string —
    RUN_TEST("out_of_range from string::at()",
        std::string s = "hello";
        char c = s.at(100);
        std::cout << c << "\n";
    )

    // — std::invalid_argument from stoi —
    RUN_TEST("invalid_argument from stoi()",
        int n = std::stoi("not_a_number");
        std::cout << n << "\n";
    )

    // — std::out_of_range from stoi overflow —
    RUN_TEST("out_of_range from stoi() overflow",
        int n = std::stoi("9999999999999999");
        std::cout << n << "\n";
    )
}
```

```

// — std::bad_alloc —
RUN_TEST("bad_alloc from new[]",
    // Request more memory than the system can provide
    long long* p = new long long[1'000'000'000'000LL];
    delete[] p;
)

// — std::bad_cast —
RUN_TEST("bad_cast from dynamic_cast on reference",
    class Base    { public: virtual ~Base() {} };
    class ChildA  : public Base {};
    class ChildB  : public Base {};
    Base* b = new ChildA();
    // dynamic_cast on reference throws bad_cast on failure
    // (pointer version returns nullptr instead – no throw)
    ChildB& ref = dynamic_cast<ChildB&>(*b);
    delete b;
)

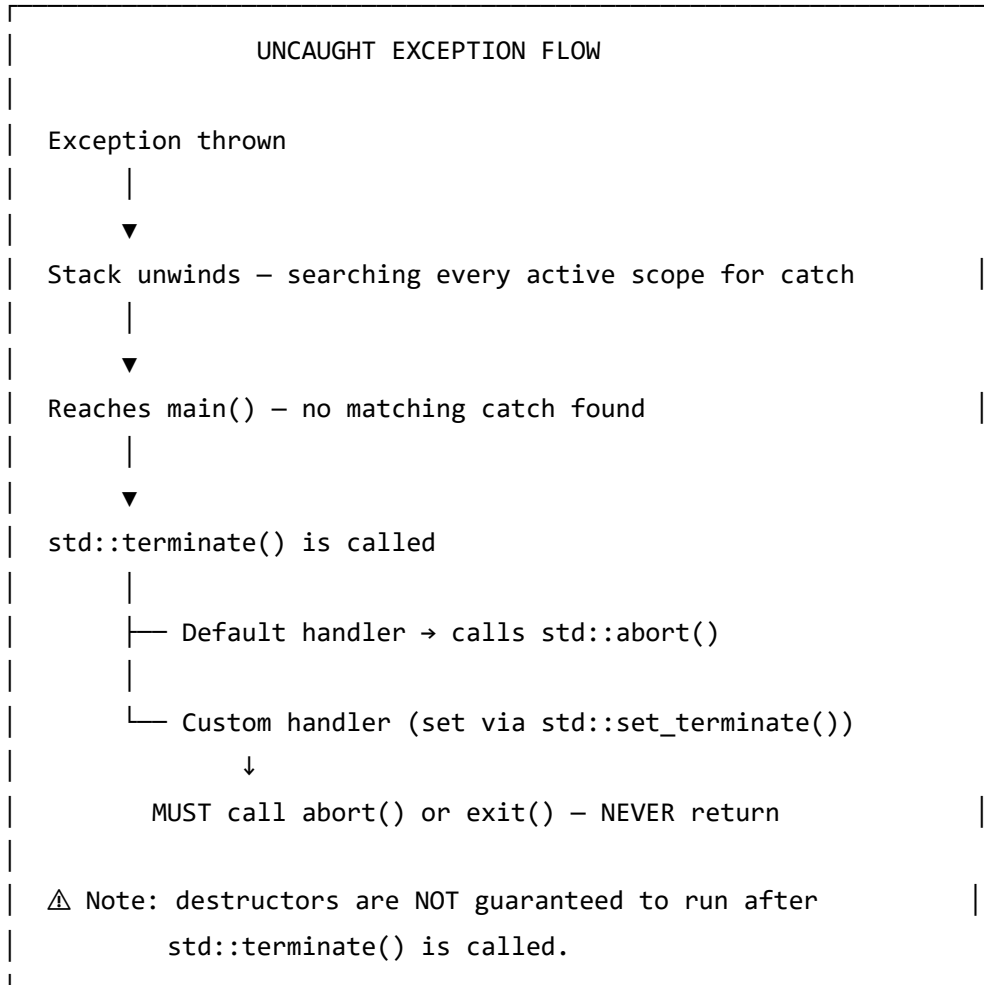
// — std::length_error —
RUN_TEST("length_error from string resize",
    std::string s;
    s.resize(s.max_size() + 1); // Request size larger than max
)

std::cout << "\n--- All standard exception demos complete ---\n";
return 0;
}

```

19.4 Uncaught Exceptions

Flow Diagram



Example 7 — Custom terminate Handler

```
#include <iostream>
#include <exception>
#include <stdexcept>
#include <cstdlib>    // std::abort

// Our custom handler – called when an exception goes unhandled
void myTerminateHandler() {
    std::cerr << "\n" << "FATAL: Uncaught exception detected" << "\n";
    std::cerr << "FATAL: Uncaught exception detected" << "\n";
    std::cerr << "FATAL: Uncaught exception detected" << "\n";

    // Try to identify what exception caused this
    std::exception_ptr ep = std::current_exception();
    if (ep) {
        try {
            std::rethrow_exception(ep); // Re-throw to catch and inspect
        }
        catch (const std::exception& e) {
            std::cerr << "  Type   : std::exception or derived\n";
            std::cerr << "  Message: " << e.what() << "\n";
        }
        catch (const int& n) {
            std::cerr << "  Type   : int\n";
            std::cerr << "  Value  : " << n << "\n";
        }
        catch (...) {
            std::cerr << "  Type   : unknown\n";
        }
    }

    std::cerr << "  Terminating program.\n";
    std::abort(); // MUST call abort or exit – never return!
}

void functionThatThrows() {
    throw std::runtime_error("This exception was never caught!");
}

int main() {
    // Install the custom terminate handler
    std::set_terminate(myTerminateHandler);
```

```
std::cout << "About to call a function that throws without a try block...\n";

functionThatThrows();    // ← No try/catch around this!

std::cout << "This line never executes.\n";
return 0;
}
```

Example 8 — noexcept Specifier

```
#include <iostream>
#include <stdexcept>
#include <utility>    // std::move

// noexcept: this function guarantees it NEVER throws.
// If it does throw, std::terminate() is called immediately.
// Benefit: compiler can optimize more aggressively.
double absoluteValue(double x) noexcept {
    return x < 0.0 ? -x : x;
}

// noexcept with a condition – only noexcept if the condition is true
template<typename T>
void safeSwap(T& a, T& b)
    noexcept(noexcept(T(std::move(a)))) // noexcept if T's move ctor is noexcept
{
    T temp = std::move(a);
    a      = std::move(b);
    b      = std::move(temp);
}

// Destructor: ALWAYS implicitly noexcept
// Throwing from a destructor during stack unwinding calls terminate()!
class Resource {
public:
    ~Resource() noexcept {    // Explicit noexcept is good practice
        // Never throw here – use a try/catch internally if you must
        try {
            // cleanup code...
        }
        catch (...) {
            // Swallow exceptions in destructor – log them instead
            std::cerr << "[Resource destructor] cleanup error (suppressed)\n";
        }
    }
};

int main() {
    // noexcept check at compile time
    std::cout << "absoluteValue is noexcept: "
        << noexcept(absoluteValue(1.0)) << "\n";    // 1 (true)
```

```
std::cout << "|-3.14| = " << absoluteValue(-3.14) << "\n";
```

```
int x = 10, y = 20;
```

```
safeSwap(x, y);
```

```
std::cout << "After swap: x=" << x << ", y=" << y << "\n";
```

```
return 0;
```

```
}
```

Example 9 — Full Exception Handling System

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
#include <fstream>

// — Custom exceptions —
class ConfigException : public std::runtime_error {
public: ConfigException(const std::string& m) : std::runtime_error(m) {} };
class ValidationException : public std::logic_error {
public: ValidationException(const std::string& m) : std::logic_error(m) {} };

// — Config parser —
struct AppConfig {
    std::string serverHost;
    int         serverPort;
    int         maxClients;
};

AppConfig loadConfig(const std::string& filename) {
    std::ifstream file(filename);
    if (!file.is_open()) {
        throw ConfigException("Config file not found: " + filename);
    }

    AppConfig config;
    config.serverHost = "localhost";
    config.serverPort = 8080;
    config.maxClients = 100;

    std::string line;
    while (std::getline(file, line)) {
        if (line.empty() || line[0] == '#') continue;
        auto eq = line.find('=');
        if (eq == std::string::npos) continue;

        std::string key   = line.substr(0, eq);
        std::string value = line.substr(eq + 1);

        if (key == "server_host") config.serverHost = value;
        else if (key == "server_port") config.serverPort = std::stoi(value);
    }
}
```



```

        else if (key == "max_clients") config.maxClients = std::stoi(value);
    }
    return config;
}

```

```

void validateConfig(const AppConfig& cfg) {
    if (cfg.serverHost.empty()) {
        throw ValidationException("server_host cannot be empty");
    }
    if (cfg.serverPort < 1 || cfg.serverPort > 65535) {
        throw ValidationException(
            "server_port must be 1-65535, got: " +
            std::to_string(cfg.serverPort));
    }
    if (cfg.maxClients < 1 || cfg.maxClients > 10000) {
        throw ValidationException(
            "max_clients must be 1-10000, got: " +
            std::to_string(cfg.maxClients));
    }
}

```

```

void startServer(const AppConfig& cfg) {
    std::cout << "  Server starting on " << cfg.serverHost
               << ":" << cfg.serverPort << "\n";
    std::cout << "  Max clients: " << cfg.maxClients << "\n";
    std::cout << "  Server running!\n";
}

```

// — Application entry point with full exception handling —

```

int main() {
    // Create a valid config file for demo
    {
        std::ofstream cfg("app.conf");
        cfg << "# Application Configuration\n";
        cfg << "server_host=0.0.0.0\n";
        cfg << "server_port=9090\n";
        cfg << "max_clients=250\n";
    }
}

```

// — Test 1: Valid config —

```

std::cout << "--- Test 1: Valid configuration ---\n";
try {
    AppConfig cfg = loadConfig("app.conf");
}

```

```

        validateConfig(cfg);
        startServer(cfg);
    }
    catch (const ConfigException& e) {
        std::cerr << " Config error: " << e.what() << "\n";
        return 1;
    }
    catch (const ValidationException& e) {
        std::cerr << " Validation error: " << e.what() << "\n";
        return 1;
    }
    catch (const std::exception& e) {
        std::cerr << " Unexpected error: " << e.what() << "\n";
        return 1;
    }
}

// — Test 2: Missing config file —
std::cout << "\n--- Test 2: Missing config file ---\n";
try {
    AppConfig cfg = loadConfig("nonexistent.conf");
    validateConfig(cfg);
    startServer(cfg);
}
catch (const ConfigException& e) {
    std::cerr << " Config error: " << e.what() << "\n";
    std::cout << " Using default configuration instead.\n";
}

// — Test 3: Invalid port in config —
{
    std::ofstream badCfg("bad.conf");
    badCfg << "server_port=99999\n"; // Invalid port
}
std::cout << "\n--- Test 3: Invalid port ---\n";
try {
    AppConfig cfg = loadConfig("bad.conf");
    validateConfig(cfg);
    startServer(cfg);
}
catch (const ValidationException& e) {
    std::cerr << " Validation error: " << e.what() << "\n";
}

```

```
    return 0;  
}
```

⚠ Common Mistakes — Exception Handling

Mistake	Problem	Correct Approach
<code>catch (T e)</code> by value	Object slicing — derived info lost	Always <code>catch (const T& e)</code>
<code>catch(...)</code> placed first	Catches everything — hides specific errors	Put <code>catch(...)</code> LAST
Throwing from destructors	<code>std::terminate()</code> if already unwinding	Mark destructors <code>noexcept</code> ; swallow internally
Empty catch blocks	Silently swallows errors — hard to debug	At minimum, log the exception
Exceptions for normal flow	Expensive; bad design	Use exceptions only for genuinely exceptional cases
Catching base before derived	Derived never matched	Always catch most-derived first

💡 Pro Tips — Exception Handling

Tip 1: Catch by `const` reference always: `catch (const std::exception& e)` . It's more efficient and prevents slicing.

Tip 2: Derive all custom exceptions from `std::exception` — allows `catch (const std::exception&)` to serve as a universal fallback.

Tip 3: Mark destructors `noexcept` . If cleanup code must do something risky, wrap it in `try/catch` inside the destructor.

Tip 4: Mark move constructors/operators `noexcept` . STL containers (like `std::vector`) can only use move semantics if moves are guaranteed non-throwing.



Quick Reference — Exception Handling

```
// throw
throw std::runtime_error("message");
throw MyException("detail", code);
throw; // Re-throw current exception (bare throw)

// try / catch structure
try { /* risky code */ }
catch (const std::invalid_argument& e) { /* specific */ }
catch (const std::exception& e)      { /* any std */ }
catch (...)                          { /* anything */ }

// Custom exception (recommended pattern)
class MyEx : public std::exception {
    std::string msg_;
public:
    MyEx(std::string m) : msg_(std::move(m)) {}
    const char* what() const noexcept override { return msg_.c_str(); }
};

// noexcept
void f() noexcept { /* never throws */ }
bool safe = noexcept(f()); // compile-time check

// terminate
std::set_terminate(myHandler); // Install custom handler
std::terminate();              // Call directly (emergency stop)
```



Exercise — Exception Handling

Problem: Create a `BankAccount` class with `deposit()`, `withdraw()`, and `transfer()` methods. Use custom exceptions for overdraft, negative amounts, and transfer to a null account. Write a `main()` that tests all error paths.

Solution:

```

#include <iostream>
#include <stdexcept>
#include <string>

// — Custom exceptions —
class BankException : public std::exception {
    std::string msg_;
public:
    BankException(std::string m) : msg_(std::move(m)) {}
    const char* what() const noexcept override { return msg_.c_str(); }
};

class OverdraftException : public BankException {
    double balance_, attempted_;
public:
    OverdraftException(double bal, double amt)
        : BankException("Overdraft: tried to withdraw $" +
                        std::to_string(amt) + " but balance is $" +
                        std::to_string(bal))
        , balance_(bal), attempted_(amt) {}
    double balance() const { return balance_; }
    double attempted() const { return attempted_; }
};

class NegativeAmountException : public BankException {
public:
    NegativeAmountException(const std::string& op, double amt)
        : BankException(op + " amount must be positive, got: $" +
                        std::to_string(amt)) {}
};

// — BankAccount class —
class BankAccount {
    std::string owner_;
    double balance_;

public:
    BankAccount(std::string owner, double initialBalance = 0.0)
        : owner_(std::move(owner)), balance_(initialBalance) {}

    void deposit(double amount) {
        if (amount <= 0) {
            throw NegativeAmountException("Deposit", amount);
        }
    }
};

```

```

    }
    balance_ += amount;
    std::cout << "    Deposited $" << amount
                << " to " << owner_
                << " (new balance: $" << balance_ << ")\n";
}

void withdraw(double amount) {
    if (amount <= 0) {
        throw NegativeAmountException("Withdrawal", amount);
    }
    if (amount > balance_) {
        throw OverdraftException(balance_, amount);
    }
    balance_ -= amount;
    std::cout << "    Withdrew $" << amount
                << " from " << owner_
                << " (new balance: $" << balance_ << ")\n";
}

void transfer(double amount, BankAccount* target) {
    if (!target) {
        throw BankException("Transfer target account is null");
    }
    if (amount <= 0) {
        throw NegativeAmountException("Transfer", amount);
    }
    // Strong guarantee: withdraw first – if it throws, target not touched
    withdraw(amount);           // May throw OverdraftException
    target->deposit(amount);     // May throw – but in practice, deposit rarely throws
    std::cout << "    Transferred $" << amount
                << " from " << owner_ << " to " << target->owner() << "\n";
}

double    balance() const { return balance_; }
const std::string& owner() const { return owner_; }
};

// — Test runner helper —
void runTest(const std::string& label, std::function<void()> test) {
    std::cout << "\n=== " << label << " ===\n";
    try {
        test();
    }
}

```

```

        std::cout << " OK\n";
    }
    catch (const OverdraftException& e) {
        std::cerr << " [OVERDRAFT] " << e.what() << "\n";
    }
    catch (const NegativeAmountException& e) {
        std::cerr << " [NEG AMOUNT] " << e.what() << "\n";
    }
    catch (const BankException& e) {
        std::cerr << " [BANK ERROR] " << e.what() << "\n";
    }
}

int main() {
    BankAccount alice("Alice", 1000.0);
    BankAccount bob("Bob", 200.0);

    runTest("Valid deposit",
        [&]{ alice.deposit(500.0); });

    runTest("Negative deposit",
        [&]{ alice.deposit(-50.0); });

    runTest("Valid withdrawal",
        [&]{ alice.withdraw(200.0); });

    runTest("Overdraft",
        [&]{ alice.withdraw(9999.0); });

    runTest("Valid transfer",
        [&]{ alice.transfer(300.0, &bob); });

    runTest("Transfer causing overdraft",
        [&]{ alice.transfer(5000.0, &bob); });

    runTest("Transfer to null account",
        [&]{ alice.transfer(100.0, nullptr); });

    std::cout << "\n--- Final Balances ---\n";
    std::cout << " " << alice.owner() << ": $" << alice.balance() << "\n";
    std::cout << " " << bob.owner() << ": $" << bob.balance() << "\n";
}

```

```
    return 0;  
}
```

Section 20: Namespaces

20.1 Understanding Namespaces

Learning Objectives

- Understand the name collision problem that namespaces solve
- Create, extend, and use namespaces
- Master `using` declarations vs. directives
- Work with nested and anonymous namespaces
- Use namespace aliases to simplify long names

Introduction

As your C++ programs grow — and especially when you start using third-party libraries — you'll encounter a frustrating problem: two different pieces of code have the same name. Library A defines `print()`. Library B also defines `print()`. You include both: the compiler stops with "ambiguous function call."

Namespaces solve this by grouping code into named scopes. Instead of a bare `print()`, you write `LibraryA::print()` or `LibraryB::print()`. The `::` is the **scope resolution operator** — it drills into a namespace (or class) to access a member.

You've already seen the most famous C++ namespace: `std`. Every time you write `std::cout`, `std::string`, or `std::vector`, you're accessing names inside the `std` namespace — the C++ Standard Library namespace. Without namespaces, using any standard library function would risk colliding with names in your own code.

Think of namespaces like city streets: two cities can each have a "Main Street" with no confusion, because you always specify the city first.

The Name Collision Problem — Diagram

WITHOUT NAMESPACES: COLLISION

```
library_a.h: void render(Scene s) { ... }  
library_b.h: void render(Scene s) { ... } ← SAME NAME!  
  
main.cpp:    render(myScene); ← COMPILER ERROR: ambiguous
```

WITH NAMESPACES: RESOLVED

```
namespace EngineA { void render(Scene s) { ... } }  
namespace EngineB { void render(Scene s) { ... } }  
  
main.cpp:    EngineA::render(myScene); ← Unambiguous ✓  
           EngineB::render(myScene); ← Unambiguous ✓
```

Example 1 — Creating and Using Namespaces

```
#include <iostream>
#include <string>
#include <cmath>

// — Define namespace "Geometry" —
namespace Geometry {
    const double PI = 3.14159265358979;

    struct Point {
        double x, y;
        Point(double x = 0, double y = 0) : x(x), y(y) {}
    };

    double distance(const Point& a, const Point& b) {
        double dx = b.x - a.x, dy = b.y - a.y;
        return std::sqrt(dx*dx + dy*dy);
    }

    double circleArea(double radius) {
        return PI * radius * radius;
    }

    double circlePerimeter(double radius) {
        return 2.0 * PI * radius;
    }

    struct Rectangle {
        double width, height;
        double area() const { return width * height; }
        double perimeter() const { return 2 * (width + height); }
    };
}

// — Define namespace "Physics" —
namespace Physics {
    const double GRAVITY = 9.81; // m/s2
    const double SPEED_LIGHT = 299792458.0; // m/s
    const double PLANCK = 6.626e-34; // J·s

    double kineticEnergy(double mass, double velocity) {
        return 0.5 * mass * velocity * velocity;
    }
}
```

```

}

double freefallDistance(double time) {
    return 0.5 * GRAVITY * time * time;
}

double freefallVelocity(double time) {
    return GRAVITY * time;
}
}

int main() {
    // — Use Geometry namespace —
    std::cout << "=== GEOMETRY ===\n";

    Geometry::Point origin(0, 0), p1(3, 4);
    std::cout << "Distance (0,0)→(3,4): "
        << Geometry::distance(origin, p1) << "\n"; // 5.0

    double r = 7.0;
    std::cout << "Circle area (r=7): "
        << Geometry::circleArea(r) << "\n";
    std::cout << "Circle perimeter (r=7):"
        << Geometry::circlePerimeter(r) << "\n";

    Geometry::Rectangle rect{4.0, 6.0};
    std::cout << "Rectangle area (4x6): " << rect.area() << "\n";

    // — Use Physics namespace —
    std::cout << "\n=== PHYSICS ===\n";

    std::cout << "KE of 5kg at 20m/s: "
        << Physics::kineticEnergy(5.0, 20.0) << " J\n";

    double t = 3.0;
    std::cout << "Free fall distance (3s): "
        << Physics::freefallDistance(t) << " m\n";
    std::cout << "Free fall velocity (3s): "
        << Physics::freefallVelocity(t) << " m/s\n";

    std::cout << "Speed of light: " << Physics::SPEED_LIGHT << " m/s\n";
}

```

```
return 0;
```

```
}
```

Example 2 — using Declarations vs. using Directives

```
#include <iostream>
#include <string>

namespace MathLib {
    double square(double x) { return x * x; }
    double cube(double x)    { return x * x * x; }
    double reciprocal(double x) { return x != 0 ? 1.0/x : 0.0; }
    const double E = 2.71828182845904;
}

namespace StringLib {
    std::string repeat(const std::string& s, int n) {
        std::string result;
        for (int i = 0; i < n; i++) result += s;
        return result;
    }
    std::string toUpper(std::string s) {
        for (char& c : s) c = std::toupper(c);
        return s;
    }
}

int main() {
    // — Option A: Fully qualified — always safe, always unambiguous —
    std::cout << "=== Option A: Fully Qualified ===\n";
    std::cout << "square(5):    " << MathLib::square(5) << "\n";
    std::cout << "toUpper(hi): " << StringLib::toUpper("hello") << "\n";

    // — Option B: using declaration — imports ONE specific name —
    std::cout << "\n=== Option B: using Declaration (one name) ===\n";
    {
        using MathLib::square;          // Only 'square' is imported into this scope
        using MathLib::E;               // Only 'E' is imported

        std::cout << "square(7):    " << square(7) << "\n";    // OK
        std::cout << "E:            " << E << "\n";            // OK
        // cube(3.0); ← ERROR: 'cube' was NOT imported
        std::cout << "cube via full: " << MathLib::cube(3) << "\n"; // Still works
    }
    // 'square' and 'E' no longer directly accessible outside the block
}
```

```

// — Option C: using directive – imports ALL names —
std::cout << "\n=== Option C: using Directive (all names) ===\n";
{
    using namespace StringLib;    // ALL of StringLib now visible

    std::cout << "repeat('ab',3): " << repeat("ab", 3) << "\n"; // OK
    std::cout << "toUpper(world): " << toUpper("world") << "\n"; // OK
}

// — Option D: using namespace std – (common but use carefully!) —
std::cout << "\n=== Option D: using namespace std ===\n";
{
    using namespace std;
    // Now you can write 'cout' instead of 'std::cout', 'string' instead
    // of 'std::string', etc. – but BEWARE: can cause hidden conflicts!
    string greeting = "Hello without std:: prefix!";
    cout << greeting << "\n";
}

return 0;
}

```

Example 3 — Nested Namespaces

```
#include <iostream>
#include <string>

// — Traditional nested namespace syntax —
namespace Company {
    namespace Backend {
        namespace Database {
            void connect(const std::string& dsn) {
                std::cout << "[DB] Connecting to: " << dsn << "\n";
            }
            void query(const std::string& sql) {
                std::cout << "[DB] Executing: " << sql << "\n";
            }
        }
        namespace Cache {
            void set(const std::string& key, const std::string& val) {
                std::cout << "[Cache] SET " << key << " = " << val << "\n";
            }
            std::string get(const std::string& key) {
                std::cout << "[Cache] GET " << key << "\n";
                return "cached_value";
            }
        }
    }
}

namespace Frontend {
    void render(const std::string& page) {
        std::cout << "[UI] Rendering: " << page << "\n";
    }
}

// — C++17 shorthand: nested namespace with :: —
namespace Company::Backend::Auth {
    bool authenticate(const std::string& token) {
        std::cout << "[Auth] Validating token: " << token << "\n";
        return !token.empty();
    }
}

int main() {
    // Full qualification
```

```
Company::Backend::Database::connect("postgresql://localhost:5432/mydb");
Company::Backend::Database::query("SELECT * FROM users WHERE active=1");
Company::Backend::Cache::set("user:42", "Alice Johnson");
Company::Backend::Cache::get("user:42");
Company::Frontend::render("dashboard");
Company::Backend::Auth::authenticate("jwt_token_abc123");
```

```
// — Namespace alias for deeply nested names —
```

```
namespace DB      = Company::Backend::Database;
namespace Cache   = Company::Backend::Cache;
namespace Auth    = Company::Backend::Auth;
namespace UI      = Company::Frontend;
```

```
std::cout << "\n--- Using aliases ---\n";
DB::connect("mysql://localhost/app");
Cache::set("session:xyz", "active");
Auth::authenticate("bearer_xyz789");
UI::render("login_page");
```

```
return 0;
```

```
}
```


Example 4 — Anonymous (Unnamed) Namespaces

```
#include <iostream>
#include <string>

// — Anonymous namespace —
// Everything inside has INTERNAL LINKAGE – visible only in THIS .cpp file.
// This is the modern replacement for 'static' at file scope.
namespace {
    // This variable and function cannot be accessed from other .cpp files
    int internalCallCount = 0;

    void logInternal(const std::string& msg) {
        internalCallCount++;
        std::cout << " [Internal #" << internalCallCount << "]" << msg << "\n";
    }

    const double INTERNAL_SCALING_FACTOR = 1.618033988; // Golden ratio
}

// Public functions CAN use anonymous namespace internals
void processData(const std::string& data) {
    logInternal("processData called with: " + data); // Uses internal function
    double result = data.length() * INTERNAL_SCALING_FACTOR;
    std::cout << " Processed length: " << result << "\n";
}

void resetSystem() {
    logInternal("System reset requested");
    internalCallCount = 0; // Reset internal counter directly
    logInternal("Counter reset to 0");
}

int main() {
    // You CAN use anonymous namespace contents directly in the same file:
    std::cout << "Direct usage from main:\n";
    logInternal("Called from main");
    std::cout << "Scaling factor: " << INTERNAL_SCALING_FACTOR << "\n\n";

    // Normal usage via public functions:
    std::cout << "Via public functions:\n";
    processData("hello world");
    processData("namespace example");
}
```

```

resetSystem();
processData("after reset");

std::cout << "\nFinal call count: " << internalCallCount << "\n";
return 0;
}

```

20.2 Rules of Namespaces

Rules Summary

NAMESPACE RULES	
R1	Namespaces can span multiple files (open/re-opened)
R2	Namespaces can be nested to any depth
R3	The global namespace is unnamed (accessible via ::name)
R4	NEVER put 'using namespace' in a header file
R5	Anonymous namespace = file-local (like static for files)
R6	Namespace aliases simplify deeply nested names
R7	inline namespace enables API versioning
R8	ADL: arguments' namespace is searched automatically

Example 5 — Namespace Extension Across Scopes

```
#include <iostream>
#include <cmath>
#include <string>

// — FIRST block: declare types —
// (in a real project, this would be in "shapes_types.h")
namespace Shapes {
    const double PI = 3.14159265358979;

    struct Circle    { double radius; };
    struct Rectangle { double width, height; };
    struct Triangle  { double a, b, c; }; // three sides

    // Declare functions (definitions come later)
    double area(const Circle& c);
    double area(const Rectangle& r);
    double area(const Triangle& t);
    double perimeter(const Circle& c);
    double perimeter(const Rectangle& r);
    double perimeter(const Triangle& t);
}

// — SECOND block: implement functions —
// (in a real project, this would be in "shapes.cpp")
// Reopening the SAME namespace — C++ merges them.
namespace Shapes {
    double area(const Circle& c) {
        return PI * c.radius * c.radius;
    }
    double area(const Rectangle& r) {
        return r.width * r.height;
    }
    double area(const Triangle& t) {
        // Heron's formula
        double s = (t.a + t.b + t.c) / 2.0;
        return std::sqrt(s * (s-t.a) * (s-t.b) * (s-t.c));
    }
    double perimeter(const Circle& c) {
        return 2.0 * PI * c.radius;
    }
    double perimeter(const Rectangle& r) {
```

```

        return 2.0 * (r.width + r.height);
    }
    double perimeter(const Triangle& t) {
        return t.a + t.b + t.c;
    }
}

int main() {
    Shapes::Circle    circle{5.0};
    Shapes::Rectangle rect{4.0, 6.0};
    Shapes::Triangle  tri{3.0, 4.0, 5.0}; // Right triangle

    std::cout << "=== Shape Calculations ===\n";
    std::cout << "Circle    area: " << Shapes::area(circle)
        << "    perimeter: " << Shapes::perimeter(circle) << "\n";
    std::cout << "Rectangle area: " << Shapes::area(rect)
        << "    perimeter: " << Shapes::perimeter(rect) << "\n";
    std::cout << "Triangle  area: " << Shapes::area(tri)
        << "    perimeter: " << Shapes::perimeter(tri) << "\n";

    return 0;
}

```

Example 6 — inline Namespaces for API Versioning

```
#include <iostream>
#include <string>

// — Library versioning with inline namespaces —
namespace API {

    // Old version — still accessible via API::v1::connect()
    namespace v1 {
        void connect(const std::string& host) {
            std::cout << "[v1] Connecting to " << host
                << " (legacy, plain HTTP)\n";
        }
        std::string query(const std::string& sql) {
            std::cout << "[v1] Running: " << sql << "\n";
            return "legacy_result";
        }
    }

    // Current version — 'inline' makes it the default
    // API::connect() automatically uses v2::connect()
    inline namespace v2 {
        void connect(const std::string& host, int port = 443,
            bool useSSL = true) {
            std::cout << "[v2] Connecting to " << host << ":" << port
                << (useSSL ? " (TLS)" : " (plain)") << "\n";
        }
        std::string query(const std::string& sql,
            bool usePrepared = true) {
            std::cout << "[v2] Running" << (usePrepared ? " (prepared): " : ": ")
                << sql << "\n";
            return "v2_result";
        }
    }
}

int main() {
    std::cout << "=== Default (inline v2) usage ===\n";
    API::connect("db.example.com");           // Uses v2 (inline)
    API::connect("db.example.com", 5432);     // v2 with explicit port
    API::query("SELECT id FROM users");       // v2 (prepared by default)
```

```
std::cout << "\n=== Explicit v1 usage (legacy) ===\n";  
API::v1::connect("legacy-db.example.com");  
API::v1::query("SELECT * FROM old_table");
```

```
std::cout << "\n=== Explicit v2 usage ===\n";  
API::v2::connect("new-db.example.com", 5432, false);  
API::v2::query("INSERT INTO log VALUES (?)", false);
```

```
return 0;
```

```
}
```

Example 7 — Argument Dependent Lookup (ADL)

```
#include <iostream>
#include <string>

// ADL: When you call a function, C++ also searches the namespace of
// each argument's type. This is why you can write 'cout << x' without
// 'std::operator<<(cout, x)' – the compiler finds operator<< in namespace std.

namespace Serialization {
    struct JsonValue {
        std::string data;
        explicit JsonValue(std::string d) : data(std::move(d)) {}
    };

    // This operator<< is in namespace Serialization
    std::ostream& operator<<(std::ostream& os, const JsonValue& j) {
        return os << "JSON{" << j.data << "}";
    }

    // This function is also found via ADL
    void prettyPrint(const JsonValue& j) {
        std::cout << "Pretty: " << j.data << "\n";
    }
}

namespace Networking {
    struct Packet {
        int id;
        std::string payload;
    };

    // Found via ADL when any argument is a Packet
    void send(const Packet& p) {
        std::cout << "[SEND] Packet #" << p.id
                  << " payload=" << p.payload << "\n";
    }
}

int main() {
    Serialization::JsonValue jv("{\"name\":\"Alice\",\"age\":30}");

    // ADL finds operator<< in Serialization namespace because jv is JsonValue
```

```
std::cout << jv << "\n";    // No 'Serialization::' needed!

// ADL finds prettyPrint in Serialization namespace
prettyPrint(jv);              // No 'Serialization::' needed!

Networking::Packet pkt{42, "GET /api/users HTTP/1.1"};
send(pkt);                    // ADL finds send() in Networking namespace

return 0;

}
```


Example 8 — Complete Namespace Design Pattern

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

// — Production-quality namespace structure —

namespace MyApp {

    // — Types sub-namespace —
    namespace Types {
        struct User {
            int      id;
            std::string name;
            std::string email;
            bool      isAdmin;
        };

        struct Session {
            std::string token;
            int      userId;
            long long expiresAt;
        };
    }

    // — Services sub-namespace —
    namespace Services {

        namespace UserService {
            // Internal state (simulated database)
            namespace {
                std::vector<Types::User> users_ = {
                    {1, "Alice Johnson", "alice@ex.com", true},
                    {2, "Bob Smith",      "bob@ex.com",  false},
                    {3, "Carol Williams", "carol@ex.com", false},
                };
            }

            Types::User* findById(int id) {
                for (auto& u : users_) {
                    if (u.id == id) return &u;
                }
            }
        }
    }
}
```

```

    }
    return nullptr;
}

std::vector<Types::User> getAll() { return users_; }

bool makeAdmin(int id) {
    auto* u = findById(id);
    if (!u) return false;
    u->isAdmin = true;
    return true;
}

namespace AuthService {
    Types::Session createSession(const Types::User& user) {
        return {"token_" + std::to_string(user.id) + "_xyz",
                user.id,
                9999999999LL};
    }

    bool validateSession(const Types::Session& s) {
        return !s.token.empty() && s.userId > 0;
    }
}

// — Utils sub-namespace —
namespace Utils {
    void printUser(const Types::User& u) {
        std::cout << " [" << u.id << "]" "
                    << u.name << " <" << u.email << ">"
                    << (u.isAdmin ? " [ADMIN]" : "")
                    << "\n";
    }
}

int main() {
    // Aliases for cleaner code
    namespace US = MyApp::Services::UserService;
    namespace Auth = MyApp::Services::AuthService;
    namespace Util = MyApp::Utils;

```

```

std::cout << "=== All Users ===\n";
for (const auto& u : US::getAll()) {
    Util::printUser(u);
}

std::cout << "\n=== Login as Bob (ID=2) ===\n";
auto* bob = US::findById(2);
if (bob) {
    auto session = Auth::createSession(*bob);
    std::cout << "  Session: " << session.token << "\n";
    std::cout << "  Valid:   " << Auth::validateSession(session) << "\n";
}

std::cout << "\n=== Make Bob an Admin ===\n";
US::makeAdmin(2);
for (const auto& u : US::getAll()) {
    Util::printUser(u);
}

return 0;
}

```

Common Mistakes — Namespaces

Mistake	Problem	Correct Approach
using namespace std in a header	ALL files including that header are polluted	Never use using namespace in headers
using namespace X at global scope in .cpp	Causes hidden ambiguity, hard-to-find bugs	Limit to function scope or at least past all includes
Forgetting :: for global scope	Unintentionally calls the wrong function	Use ::functionName to explicitly call global scope
Using anonymous namespace in headers	Defeats the purpose — creates a copy per TU	Anonymous namespaces belong only in .cpp files

Mistake	Problem	Correct Approach
Deep nesting without aliases	Verbose, unreadable code	Create namespace aliases: <code>namespace DB = App::Backend::DB</code>

Pro Tips — Namespaces

Tip 1: The C++ Core Guidelines say: "Never write `using namespace std` in a header file." This is one of the most commonly violated rules in beginner code.

Tip 2: Use anonymous namespaces (not `static`) for file-private helpers. They work for functions, variables, AND types — `static` only works for functions and variables.

Tip 3: Use `inline namespace` for library versioning — it lets you add a new default version without breaking existing code that uses fully qualified names.

Tip 4: Short, meaningful namespace names (3–10 characters) are better for readability. Use aliases for third-party libraries with long names.

Quick Reference — Namespaces

```
// Define
namespace Foo { void f(); int x; }

// Extend (reopen – merges with previous definition)
namespace Foo { void g(); }

// Access
Foo::f();    Foo::x;

// using declaration (one name)
using Foo::f;    f();    // now works without Foo::

// using directive (all names – avoid in headers!)
using namespace Foo;    f(); g(); x;

// Alias
namespace F = Foo;    F::f();

// Anonymous (file-local)
namespace { void hidden(); }

// Nested (traditional)
namespace A { namespace B { void h(); } }
A::B::h();

// Nested (C++17 shorthand)
namespace A::B::C { void i(); }

// Inline (versioning)
namespace Lib { inline namespace v3 { void api(); } }
Lib::api();    // uses v3 automatically
Lib::v3::api(); // explicit

// Global scope
::globalFunction();    // Use :: to access global namespace explicitly
```

Exercise — Namespaces

Problem: Design a logging system using namespaces. Create a `Logger` namespace with `Console` and `File` sub-namespaces. Each should implement `log(level, message)`. Write a main function that

tests both loggers and uses a `using` declaration to simplify calls.

Solution:

```

#include <iostream>
#include <fstream>
#include <string>
#include <ctime>

namespace Logger {

    // — Shared types —
    enum class Level { DEBUG, INFO, WARNING, ERROR, CRITICAL };

    namespace {
        // Internal helper – not visible outside this file
        std::string levelToString(Level lvl) {
            switch (lvl) {
                case Level::DEBUG:    return "DEBUG";
                case Level::INFO:     return "INFO ";
                case Level::WARNING:  return "WARN ";
                case Level::ERROR:    return "ERROR";
                case Level::CRITICAL: return "CRIT ";
            }
            return "?????";
        }

        std::string timestamp() {
            std::time_t now = std::time(nullptr);
            char buf[24];
            std::strftime(buf, sizeof(buf), "%H:%M:%S",
                          std::localtime(&now));
            return buf;
        }
    }

    // — Console sub-namespace —
    namespace Console {
        // Minimum level to print (filter out noisier levels)
        static Level minLevel = Level::DEBUG;

        void setMinLevel(Level lvl) { minLevel = lvl; }

        void log(Level level, const std::string& message) {
            if (level < minLevel) return;

            // Color codes (ANSI – work on Linux/macOS/Windows 10+)

```

```

const char* color = "";
const char* reset = "\033[0m";
switch (level) {
    case Level::DEBUG:    color = "\033[36m"; break; // Cyan
    case Level::INFO:     color = "\033[32m"; break; // Green
    case Level::WARNING:  color = "\033[33m"; break; // Yellow
    case Level::ERROR:    color = "\033[31m"; break; // Red
    case Level::CRITICAL: color = "\033[35m"; break; // Magenta
}

std::cout << color
    << "[" << timestamp() << "]" "
    << "[" << levelToString(level) << "]" "
    << message
    << reset << "\n";
}
}

// — File sub-namespace —
namespace File {
    static std::string logFilePath = "app.log";
    static Level      minLevel     = Level::INFO; // Files default to INFO+

    void setFile(const std::string& path) { logFilePath = path; }
    void setMinLevel(Level lvl)          { minLevel = lvl; }

    void log(Level level, const std::string& message) {
        if (level < minLevel) return;

        // ios::app – append to existing log file
        std::ofstream out(logFilePath, std::ios::app);
        if (!out) {
            std::cerr << "Logger::File: cannot write to " << logFilePath << "\n";
            return;
        }

        out << "[" << timestamp() << "]" "
            << "[" << levelToString(level) << "]" "
            << message << "\n";
    }
}

// — Broadcast: log to ALL backends —

```



```

void logAll(Level level, const std::string& message) {
    Console::log(level, message);
    File::log(level, message);
}
}

int main() {
    // — Configure loggers —
    Logger::File::setFile("system.log");
    Logger::Console::setMinLevel(Logger::Level::DEBUG); // Show all in console
    Logger::File::setMinLevel(Logger::Level::WARNING); // Only warnings+ to file

    // — Test console logging —
    std::cout << "=== Console Logger (all levels) ===\n";
    Logger::Console::log(Logger::Level::DEBUG, "Initializing subsystems");
    Logger::Console::log(Logger::Level::INFO, "Server started on port 8080");
    Logger::Console::log(Logger::Level::WARNING, "Memory usage above 80%");
    Logger::Console::log(Logger::Level::ERROR, "Failed to connect to cache");
    Logger::Console::log(Logger::Level::CRITICAL, "Database connection lost!");

    // — using declaration for cleaner code —
    std::cout << "\n=== File Logger (WARNING+ only) ===\n";
    using Logger::File::log;
    using Logger::Level;

    log(Level::DEBUG, "This won't appear in the file (below minLevel)");
    log(Level::INFO, "This also won't appear in the file");
    log(Level::WARNING, "Disk space below 10% – this WILL appear");
    log(Level::ERROR, "Plugin load failed – this WILL appear");
    log(Level::CRITICAL, "Out of memory – this WILL appear");
    std::cout << "Wrote warnings/errors to system.log\n";

    // — Broadcast —
    std::cout << "\n=== Broadcast (both backends) ===\n";
    Logger::logAll(Level::CRITICAL, "SYSTEM SHUTDOWN INITIATED");

    // — Show file contents —
    std::cout << "\n=== Contents of system.log ===\n";
    std::ifstream logFile("system.log");
    if (logFile.is_open()) {
        std::string line;
        while (std::getline(logFile, line)) {
            std::cout << " " << line << "\n";
        }
    }
}

```

```

    }
}

return 0;
}

```



Master Summary

SECTIONS 18-20 QUICK REFERENCE CHEAT SHEET

FILE I/O (#include <fstream>)

- ofstream → write/create ifstream → read fstream → both
- Always check is_open() Always close() or use RAII scope
- getline() for lines >> for tokens read() for binary
- seekg/seekp move pointers tellg/tellp query position
- State: good() eof() fail() bad() clear() resets flags

EXCEPTION HANDLING (#include <stdexcept>)

- throw ExType("msg") → creates + throws exception
- try { } catch (const T& e) { } catch (...) last
- Derive custom exceptions from std::exception
- Stack unwinding calls destructors → RAII is exception-safe
- throw; (bare) re-throws current exception
- noexcept marks no-throw functions; violating calls terminate()
- set_terminate() installs custom fatal error handler

NAMESPACES

- namespace Foo { } groups names into a scope
- Foo::name accesses members (scope resolution operator)
- using Foo::name imports one name
- using namespace Foo imports all (NEVER in headers!)
- namespace F = Foo creates an alias
- namespace { } is anonymous (file-private)
- inline namespace v2 { } makes v2 the default version



Additional Resources

Topic	Resource
File I/O reference	cppreference.com — File I/O
Exception handling	cppreference.com — Exceptions
Namespaces	C++ Core Guidelines — Namespaces
Standard exceptions	<code><stdexcept></code> header reference at cppreference.com
Books	<i>Effective C++</i> by Scott Meyers — Items 8, 29–32
Books	<i>C++ Primer (5th Ed.)</i> by Lippman, Lajoie, Moo
Books	<i>A Tour of C++</i> by Bjarne Stroustrup (concise, modern)
Practice	HackerRank C++ domain
Practice	LeetCode — C++ problems
Community	cplusplus.com forums

C++ Comprehensive Training Guide — Sections 18–20

C++ Standard: C++17 (examples compatible with C++14 where noted)

Compiled with: `g++ -std=c++17 -Wall -Wextra`